Counter Cultures — Catalog Content + Launch Readiness Plan

Prepared 2026-05-28 (v4, audit-grounded, POC-gated, no forecasted weeks) · Working backwards from a real revenue-generating storefront

> **v4 is intentionally smaller than v3.** v1 was a vibe. v2 was code-grounded for findings but assumed pipelines worked at scale. v3 added POC gates but still predicted weeks 2–6 against unproven plumbing. **v4 commits only to what is verifiable today and to a Week 1 POC plan with explicit acceptance criteria. Weeks 2–6 are deliberately not scheduled in this doc; they get written after the POC results are in.**
>
> Tag legend used inline: - **PROVEN** — exercised by a passing test, or a verified artifact on disk (file we inspected). - **GROUNDED** — code path read; not yet exercised or run end to end. Treat as a hypothesis until POC. - **UNVERIFIED** — depends on external systems we cannot inspect from a code audit (live Odoo state, live Squarespace, third-party PAC, Cloudflare account state, vendor exports, Roger's bandwidth). Must be POC'd before any commitment.

---

1. What is PROVEN today (no further audit needed)

Every line below is anchored to code we read, or a passing test, or a file on disk we inspected. These are facts.

Catalog data plumbing

- **PROVEN.** `CC_Products_Full` is the catalog read source. `app/lib/products-full.ts:46` reads `GOOGLE_SHEETS_ID_PRODUCTS_FULL`.
- **PROVEN.** The hourly `odoo-sync` cron syncs invoices, payments, sale orders, purchase orders, partners. `app/lib/odoo/sync.ts:80–177`. **It does NOT sync products.** A products sync from Odoo does not exist.
- **PROVEN.** `netlify.toml` cron list: `stale-deal-sweep` (0 14 * * *), `odoo-sync` (0 * * * *), `fx-sync` (0 17 * * *), `keepalive` (/3 * * * *). No product-sync cron.

Content plumbing — AI descriptions

- **PROVEN.** Generator exists at `app/api/dashboard/products/generate-description/route.ts` and `app/lib/product-descriptions.ts`. Uses Anthropic Haiku 4.5.
- **PROVEN, IMPORTANT.** The generator is **per-product, on-demand, NOT batch**. File comment: "*Generation is on-demand, not batch — only products Roger actually opens get a description.*"
- **PROVEN, IMPORTANT.** Approval is **sheet-editing, no UI.** Same file: "*Roger flips it to 'approved' by editing the sheet directly.*"
- **PROVEN, CRITICAL.** AI descriptions are not in the PDP resolver. `app/lib/pdp-description.ts` 5-step chain has no Product_Descriptions read. Approved AI descriptions do not reach customers today.

Content plumbing — scrape

- **PROVEN it ran once.** `app/lib/product-content.json` is 1.6 MB, 1,550 entries, dated 2026-05-12.
- **PROVEN it ran partially.** Spot check: many entries have descriptions but empty `title`, `gallery`, `breadcrumb`. So the pipeline produces partial output on the data it last ran against.
- **GROUNDED.** 14 numbered scripts under `scripts/scrape/` (`01-cc-mx-sitemap.ts` … `12-final-audit.ts`). End-to-end fitness against current live Squarespace is **UNVERIFIED**.

Content plumbing — images

- **PROVEN.** 4,236 thumbnails, 96 gallery folders, 468 spec PDFs (per `docs/data-sources-of-truth.md:97`, itself a code-verified audit).
- **UNVERIFIED.** That the merchandised 4,236 SKUs and the 4,236 thumbnails are the same set.
- **UNVERIFIED.** Cloudflare Images + R2 approval status.

Money path

- **PROVEN.** Storefront card sale does not reach Odoo. `app/api/stripe/webhook/dispatcher.ts:310–386` writes Sheets, fires rule engine, never calls `createQuote` / `confirmAndInvoiceOrder` / `registerPayment`. The Stripe→Odoo bridge at `:152–208` fires only for portal-initiated deal payments with `metadata.odoo_invoice_id`.
- **PROVEN.** Factura is a stub. `app/lib/factura/provider.ts` `issueFactura()` returns `{ ok: false, reason: "factura_provider_not_configured" }`. Comment: "*Stub — real Facturapi integration ships in follow-up PR.*"
- **PROVEN.** Five email routes hardcode `noreply@countercultures.com.mx`: `send-quote/route.ts:14`, `share/route.ts:17`, `shop/request-quote/route.ts:38`, `invoices/[id]/notify-ups/route.ts:20`, `traficos/[id]/send-to-broker/route.ts:69`.
- **PROVEN.** Zero tests exercise `handleCartPurchaseCompleted`, `registerStripePaymentInOdoo`, or `issueFactura`.

Launch-critical infra

- **PROVEN.** `public/robots.txt` is still in the working tree: `User-agent: * Allow: /` with `Disallow: /dashboard` and `Disallow: /api` only. Untracked. One careless commit makes staging crawlable.
- **PROVEN.** No 301 redirect map. No `public/_redirects`, no `[[redirects]]` in `netlify.toml`.
- **PROVEN.** Keepalive cron runs every 3 minutes.
- **PROVEN.** Search trilogy + runtime fix shipped (`7341ec5`, `240d8d0`, `9f6c7be`, `8e3eb5c`, merge `4946b51`). 139 tests pass on `main`.

That is the verified state. Anything not on this list, treat as unproven.

---

2. The target framing (no commitment to dates)

Two phases. v3 said "Jul 6 / Jul 7." v4 says **the dates are conditional on POC outcomes in §3.** If POCs reveal a 4-week problem, the dates move. The target framing is:

- **Soft launch** — customers browse the merchandised catalog and request a quote. No live card checkout on the storefront. Sales closes manually in Odoo, Antonina emits factura manually. Achievable on the current code if the POCs in §3 pass.
- **Live card-checkout** — fast-follow phase. Cart→Odoo bridge, CFDI/factura via real PAC, products sync wired, AI descriptions wired, image CDN populated. **Cannot land on the soft-launch date** because all four require successful POCs first.

The number 4,236 (merchandised SKUs) is a planning target carried from prior docs (`LEAN-ON-ODOO-PLAN.md §7` decision). **It is UNVERIFIED until we confirm the merchandised count and that the 4,236 SKUs have thumbnails.**

---

3. Week 1 — POC plan (the only week this doc commits to)

Six POCs. Each has explicit acceptance criteria, sample size, expected time, and what happens if it fails. **Nothing past Week 1 is scheduled in this doc** because nothing past Week 1 can be honestly scheduled until POC outcomes are known.

POC-A — AI description batch viability

- **Goal:** prove the existing per-product generator can be wrapped in a batch loop and produce usable output at scale, with bounded cost.
- **Method:** write a thin script (Node or a one-off route) that loops over 10 SKUs across 3 brands, calls `generateDescription(product)` from `app/lib/product-descriptions.ts`, and verifies each row lands in the `Product_Descriptions` tab with `status="pending"`.
- **Acceptance:**
- All 10 calls return without error.
- 10 rows appear in `Product_Descriptions` with non-empty EN and ES descriptions.
- Anthropic cost for the batch matches the file's quoted ~$0.001/product (so 10 SKUs ≈ $0.01).
- Roger reads at least 3 of them and rates them "shippable to customers."
- **Estimated time:** half a day to a day.

- **Fail modes:** API rate limits, cost overruns, quality so poor Roger rejects all 10 → AI batch-fill is not a viable path; we need a manual writing plan.

POC-B — Scrape pipeline against current live Squarespace

- **Goal:** prove `scripts/scrape/01–12` runs cleanly against today's `countercultures.com.mx` and produces complete entries.
- **Method:** run the full numbered pipeline against 5 known SKUs whose Squarespace pages we can name in advance. Inspect the resulting `product-content.json` entries.
- **Acceptance:** all 5 entries have non-empty `title`, at least one `gallery` URL, `descriptionEs`, `breadcrumb`, and a `specSheetUrl` if the Squarespace page had a PDF link.
- **Estimated time:** half a day to a day.
- **Fail modes:** Squarespace HTML structure changed; rate limited; some fields are stale. → re-run the scrape isn't a viable path to fill the 4,236.

POC-C — Products sync from Odoo

- **Goal:** prove we can pull 50 product records from Odoo via API and write them into a sheet matching `CC_Products_Full`'s schema.
- **Method:** small script that authenticates via `ODOO_API_KEY`, reads 50 `product.product` records with the fields `CC_Products_Full` expects (id, name, sku, brand, category, list_price, currency, sale_ok, sat_code), writes them to a test tab. Compare schema to `CC_Products_Full` headers.
- **Acceptance:** all 50 records round-trip without schema gaps; field types match; brand strings line up with Brand Kit slugs.
- **Estimated time:** one day.
- **Fail modes:** field schema drift; permission errors; brand strings don't match Brand Kit. → a sync needs more work than a one-day cron build; replan.

POC-D — 301 redirect mapping feasibility

- **Goal:** prove we can build the Squarespace → new-site redirect map automatically rather than by hand for thousands of URLs.
- **Method:** crawl the live Squarespace sitemap for ~50 product URLs. Map each to the new-site equivalent using the existing slug pipeline (`toSlug`). Hand-verify 10 of the mappings.
- **Acceptance:** at least 45 of 50 URLs map cleanly; the 10 hand-verified ones are correct.
- **Estimated time:** one day.
- **Fail modes:** Squarespace URLs don't follow a pattern we can reverse; products on Squarespace don't exist in CC_Products_Full. → 301 map becomes manual triage, not a one-fix-file job.

POC-E — PAC sandbox emit (Phase 2 derisk)

- **Goal:** prove a real CFDI can be emitted against Counter Cultures' RFC in a PAC sandbox before Phase 2 commits to any specific vendor.
- **Method:** the Odoo specialist picks a PAC (Facturapi is one option, not the only one), provisions a sandbox, emits one test factura against the actual business RFC.
- **Acceptance:** one valid CFDI emitted, UUID returned, XML retrievable.
- **Estimated time:** depends entirely on the specialist's familiarity with Mexican PACs.
- **Fail modes:** RFC validation issues, CSD certificate setup blockers, the PAC doesn't support our product types. → Phase 2 needs a different PAC or a different approach.

POC-F — Cart→Odoo dry run (Phase 2 derisk)

- **Goal:** prove `createQuote` → `confirmAndInvoiceOrder` → `registerPayment` in `app/lib/odoo/write.ts` actually work against the live Odoo's chart of accounts, journals, tax codes, and IVA setup.
- **Method:** in a controlled session, manually invoke the chain against one test cart payload. Verify that the SO, invoice, and payment land correctly in Odoo and reconcile.
- **Acceptance:** one test SO created, confirmed, invoiced, and a payment registered against it, all visible in Odoo with correct IVA breakdown.

- **Estimated time:** one to two days (specialist plus Josh).
- **Fail modes:** journal mapping missing; tax codes wrong; IVA breakdown doesn't match the Mexican localization Odoo setup. → Phase 2 is a bigger job than a single fix file.

---

4. Independent of POCs — three small fixes that can ship in Week 1

These three do not depend on any POC, are PROVEN issues with PROVEN fix patterns, and can ship as fix files in parallel with the POCs.

#	Fix	Tag	Why it can ship without a POC
L1	Remove `public/robots.txt` (H2)	**PROVEN**	The file is in the working tree; deleting it restores the dynamic noindex behavior in `app/robots.ts`. No POC needed; verification is loading the staging URL and confirming the `X-Robots-Tag` / robots.txt response.
L2	Fix the 5 hardcoded email senders (H3)	**GROUNDED + provable in Week 1**	The fix pattern (env-based sender + redirectRecipient) is already used in `app/lib/email.ts`. Acceptance: send one test quote-by-email from `/api/shop/request-quote` after the fix; verify it arrives at the allowlisted recipient.
L3	Wire AI descriptions into the PDP resolver	**GROUNDED + provable in Week 1**	Small additive step (`step 2.5`) to `app/lib/pdp-description.ts`, reading `Product_Descriptions` with `status=approved`. Acceptance: an approved description for a single test SKU renders on its PDP after the fix. **L3 only matters if POC-A passes** (otherwise approved descriptions don't exist at volume), but L3 itself is safe to land independently.

---

5. After Week 1 — branching logic, not a schedule

This doc deliberately does not lay out Weeks 2–6. Once POC results land, the next version of this plan picks one of these branches per workstream. **Until then, no week is committed.**

| If POC-A passes | Build the batch wrapper. Start batched generation for the merchandised set. Roger approves in sheet batches. | | If POC-A fails | The merchandised set needs a manual writing plan. Either a writer, or a different generation approach. Replan content fill from scratch. | | If POC-B passes | Re-run scrape to refill `product-content.json` against the live Squarespace site. | | If POC-B fails | Fall back to the PDP resolver's `"{brand} {name}"` step for unmerchandised long-tail. Accept the visual coverage gap. | | If POC-C passes | Build the products-sync cron in week 2. | | If POC-C fails | Ship soft launch with manual product exports; document the manual cadence in the handoff runbook. | | If POC-D passes | Generate the full 301 map in week 4 from the same script scaled to all Squarespace URLs. | | If POC-D fails | Manual triage in week 4–5; accept SEO equity loss on URLs we can't map. | | If POC-E passes | Phase 2 PAC integration sized in days, not weeks. | | If POC-E fails | Phase 2 PAC pick is wrong; surface the blocker to Roger and the specialist immediately. | | If POC-F passes | Phase 2 cart→Odoo wiring is a known-effort fix file. | | If POC-F fails | Phase 2 cart→Odoo needs Odoo-config work first. Larger scope than a fix file. |

After Week 1, the writeup gets a v5 with actual scheduled weeks against the branches that survived.

---

6. What this plan deliberately does NOT commit to (yet)

- A Jul 6 launch date.
- A guarantee that the 4,236 will have AI descriptions by any specific date.
- That image migration will land on Cloudflare in any specific week.
- That Phase 2 (cart→Odoo + CFDI) will land within 6 weeks of soft launch.
- A Phase 2 PAC vendor.
- That POC-A, B, C, D, E, or F will pass.

Anything making one of those commitments without showing the underlying POC has shipped a fact is **assumption, not plan**.

---

7. Honest constraints on this doc itself

- I wrote this from a sandbox that can read code and run vitest, but cannot run live scrapes, hit Anthropic at scale, talk to Odoo in production, or test Cloudflare onboarding. Every POC above must be executed on Josh's machine (or by an agent with the relevant credentials) — not validated from the sandbox.
- The schema of `Product_Descriptions` is **GROUNDED** in code (`DescriptionRecord` interface in `app/lib/product-descriptions.ts`), but the live sheet's current state and whether it has any approved rows is **UNVERIFIED**.
- "Merchandised 4,236 SKUs" is a number carried from prior planning docs. The actual definition (which SKUs are in the set, who maintains the list) is **UNVERIFIED** in this audit. Before any batch work, that set has to be defined as a verifiable list.

---

8. The single sentence that holds up

Run the six POCs in Week 1 along with three independent fixes (L1, L2, L3). Reconvene at end of Week 1. Schedule Weeks 2–6 based on which POCs survived. Anything written today about Weeks 2–6 is forecast, not plan.